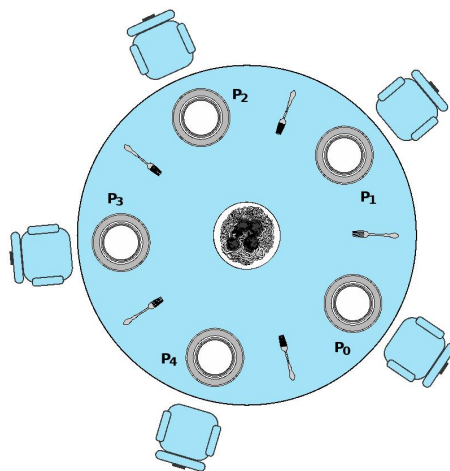


CSC 3453 - Operating Systems

Homework #3 - Dinning Philosophers' problem (150 pts)

You are allowed to work in groups of two for this assignment.

The dinning philosophers' problem is a classic synchronization problem. Though, it does not notably represent a real-world problem, it provides a significant learning value, particularly in process synchronization. It is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner. There are n philosophers dinning together at the same table. Each philosopher has their own place at the table. There is a fork between each plate. The dish served is a kind of spaghetti which has to be eaten with two forks. Each philosopher can only alternately think and eat. Moreover, a philosopher can only eat their spaghetti when they have both a left and right fork. Thus, two forks will only be available when their two nearest neighbors are thinking, not eating. After an individual philosopher finishes eating, they will put down both forks.



Write a C program (phils.c) that solves the dinning philosophers' problem where each dinning philosopher is modeled as a thread. Use semaphores (**not mutexes**) for synchronization. Your program will take two arguments: the number of philosophers at the table and a number of times to eat for each philosopher.

Care must be taken to prevent a **deadlock**. One possible solution to alleviate the deadlock is known as “asymmetric solution”, that is, **an odd philosopher picks up first a left chopstick and then the right one, while an even philosopher picks up first a right chopstick and then the left one (or vice-versa)**. Your solution should exhibit some level of fairness (A philosopher should not eat 10 times while another has not eaten a single time).

Example of output:

```
$ ./phils 7 10 # 7 philosophers, each eats 10 times total.
Philosopher 0 is hungry...
Philosopher 1 is thinking...
Philosopher 2 is hungry...
Philosopher 3 is thinking...
Philosopher 4 is hungry...
Philosopher 5 is hungry...
Philosopher 6 is hungry...
Philosopher 2 is eating...
Philosopher 0 is eating...
Philosopher 4 is eating...
Philosopher 5 is thinking...
Philosopher 6 is thinking...
...
```

Submit a single .c source file of your solution and an example output (**output.txt**). Do not forget to include all team member names.

I should be able to compile and run your code as follows (an example):

```
gcc -o phils phils.c -lpthread && ./phils 4 5
```

Main Criteria:

- The number of philosophers (therefore chopsticks) and the number of eat times should be dynamic. I will not accept a solution for a fix number of philosophers.
- As stated above, you need to use semaphores, not mutexes in your solution (not using semaphores will cause an automatic zero).
- Use the sleep() function to simulate eating and thinking (sleep time can be set to 1 or 2 seconds).
- The initial states of the philosophers should be randomized (i.e., either thinking or hungry).
- The philosophers should never start eating without displaying a thinking or hungry state.